



Operating Systems Assignment 3

ID1200

December 18, 2025

Lucas Frykman

Student Emails: lfrykman@kth.se

Question 1

(using p164 in OSTEP book)

q1(a) first-fit allocation

{P0, 115KB}: S0 (32KB too small) → S1 (100KB too small) → S2 (256KB fits) → {P0, S2} (S2 141KB remaining)

{P1, 500KB}: All partitions are too small → {P1, X}

{P2, 256KB}: S0 (32KB too small) → S1 (100KB too small) → S2 (141KB too small) reaches end of list → {P2, X}

{P3, 31KB}: S0 (32KB fits) → {P3, S0} (S0 1KB remaining)

Free memory remaining: (S0 1KB, S1 100KB, S2 141KB, S3 128 KB, S4 64KB)

q1(b) best fit allocation

{P0, 115KB}: S3 (smallest partition $115\text{KB} \leq$) → {P0, S3} (S3 13KB remaining)

{P1, 500KB}: All partitions are too small → {P1, X}

{P2, 256KB}: S2 (smallest remaining partition $256\text{KB} \leq$) → {P2, S2} (S2 0KB remaining)

{P3, 31KB}: S0 (smallest remaining partition $32\text{KB} \leq$) → {P3, S0}

Free memory remaining: (S0 1KB, S1 100KB, S2 0KB, S3 13KB, S4 64KB)

q1(c) worst fit allocation

{P0, 115KB}: S2 (largest remaining $\geq 256\text{KB}$) → {P0, S2} (S2 141KB remaining)

{P1, 500KB}: All partitions too small → {P1, X}

{P2, 256KB}: All partitions too small → {P2, X}

{P3, 31KB}: S2 (largest remaining ≥ 31) → {P3, S2} (S2 110KB remaining)

Free memory remaining: (S0 32KB, S1 100KB, S2 110KB, S3 128 KB, S4 64KB)

q1(d) measuring fragmentation

External fragmentation should be a measure of how non contiguous the total free memory is. When picking our method of allocation we might want to minimize this so that we have a higher likelihood of successfully allocating more processes in the near future. In other words, of the total free memory available we want to get as much of it as possible in to one contiguous block. We can look at the largest contiguous block available and compare it against the rest of the total free memory.

i.e $\text{fragmentation} = 1 - \frac{\text{Largest Contiguous Partition}}{\text{Total Free Memory}}$

Calculations for a,b,c:

$$a = 1 - \frac{141}{1+100+141+128+64} \approx 0.675$$

$$b = 1 - \frac{100}{1+100+0+13+64} \approx 0.438$$

$$c = 1 - \frac{128}{32+100+110+128+64} \approx 0.705$$

Best fit allocation performed the best in this example by successfully allocating most of the processes requested as well as having the least amount of fragmentation.

Question 2

Assume the logical address space consists of 1024 pages with a page size of 4 KiB. The physical memory consists of 64 frames.

q2(a)

(reference p361 in textbook)

The logical address is a address generated by the CPU.

We need the n bits for the page offset d : These bits are used to determine the page size and intra-page indexing.

$$4\text{KiB} = 4 \times 1024 = 4096 \text{ bytes} = 2^{12} \text{ bytes} \rightarrow n = 12$$

and we need $m - n$ bits for page number p : These bits tells us which page to look at.

$$1024 = 2^{10} \rightarrow m - n = 10$$

So the logical address needs $m = 22$ bits

q2(b)

To translate logical address generated by the cpu to a physical address we replace the page number p with the frame number f . In other words we need $\log_2(64) = 6$ f bits and $n = 12$ d bits. So the physical address needs $6 + 12 = 18$ bits.

q2(c)

22 bits. The virtual memry address is equivilant to the logical memory address in the context of paging.

Now, on another system with page size of 16 KiB, calculate the page numbers and offsets for each of the following logical address in decimal numbers

The decimal form of (p, d) is given by $p * \text{pageSize} + d$. This is simple to invert with just divison and taking the modulus.

16KiB = $16 \times 1024 = 16384$ bytes

a) Given logical address 189150

Page number $p = \text{floor}(189150/16384) = \text{floor}(11.54) = 11$

Offset number $d = 189150 \% 16384 = 8926$

b) Given logical address 83205425

page number $p = \text{floor}(83205425/16384) = \text{floor}(5078.45) = 5078$

Offset number $d = 83205425 \% 16384 = 7473$

Question 3

q3(a)

We know 128KiB = $128 \times 1024 = 131072$ bytes

$p_{650003} = 650003/131072 = \text{floor}(4.95) = 4$

$d_{650003} = 650003 \% 131072 = 125715$

$p_{42096} = 42096/131072 = 0$

$d_{42096} = 42096 \% 131072 = 42096$

$p_{216202} = 216202/131072 = \text{floor}(1.64) = 1$

$d_{216202} = 216202 \% 131072 = 85130$

q3(b)

(see p367 and lecture notes for calculating EAT)

For 3-level TLB we have the following hit latencies, hit ratios

L1 hit latency: 1ns, 0.98

L2 hit latency: 1ns, 0.98

L3 hit latency: 1ns, 0.98

with miss latency 100ns

The effective access time is given by

$\text{EAT} = 0.98 * 1\text{ns} + (0.02) * 0.98 * 1\text{ns} + (0.02) * (0.02) * 0.98 * 1\text{ns} + (0.02) * (0.02) * (0.02) * 100\text{ns}$
 $= 1.000792\text{ns}$

(I'm following the convention in the lecture notes where the 100ns memory access latency isn't added to each time a hit happens.)

q3(c)

(see p371)

We'll assume both page tables belong to a different system since the bit width of d is too small for 128KiB page size. It instead corresponds to a page size of 4KiB = $2^{12} = 4096$ bytes. We'll now construct the two-level page table using the following method of converting to binary then splitting the bits

$p_{650003} = 650003/4096 = 158 \rightarrow_{base2} 0000\ 0000\ 0000\ 1001\ 1110$ which gives us the split

$p_1 = (0000000000)_2 = 0$, $p_2 = (0010011110)_2 = 158$ and $d = 650003 \% 4096 = 2835$

$p_{42096} = 42096/4096 = 10 \rightarrow_{base2} 20\text{b}1010$ which gives us the split

$p_1 = (0000000000)_2 = 0$, $p_2 = (0000001010)_2 = 10$ and $d = 42096 \% 4096 = 1136$

$p_{216202} = 216202 / 4096 = 52 \rightarrow_{base2} 20b1001 \ 1110$ which gives us the split

$p_1 = (0000000000)_2 = 0$, $p_2 = (0010011110)_2 = 52$ and $d = 216202 \% 4096 = 3210$

q3(d)

$p_{650003} = 650003 / 4096 = 158 \rightarrow_{base2} 52b1001 \ 1110 \rightarrow (p_1, p_2, p_3) = (0, 0, 158) \ d = 2835$

$p_{42096} = 42096 / 4096 = 10 \rightarrow_{base2} 52b1010 \rightarrow (p_1, p_2, p_3) = (0, 0, 10) \ d = 1136$

$p_{216202} = 216202 / 4096 = 52 \rightarrow_{base2} 52b1001 \ 1110 \rightarrow (p_1, p_2, p_3) = (0, 0, 52) \ d = 3210$

question 4

12 bit system with 256 byte pagesize gives page number 4 bit width and offset 8 bit width. this means that page number is the first digit of a hex and

q4(a)

1. first find the page number and offset of the address

$0x2a1 = (673)_{10} \rightarrow (p, d) =$

$(\text{address}/\text{pagesize}, \text{address} \% \text{pagesize}) = (673/256, 673 \% 256) = (2, 161) = (0x2, 0xA1)$

2. look up $p = 2$ in the table and we find $f = 0xa$

3. construct the physical address by concatenation $f|d = 0xaa1$

q4(b)

(note : I just realized at this point that the manual division and modulus I've been doing up to this point was redundant and I could have just done all these conversions by just directly splitting up digits)

1. first find the page number and offset of the address by converting to binary then hex and split it with respect to bit widths

$0x4e6 \rightarrow (0100 \ 1110 \ 0110) \rightarrow (p, d) = (0x4, 0xe6)$

2. look up $p = 4$ in the table and we find that the entry is empty (i.e page fault). to handle this we must allocate a frame from the free list $\{9, f, d\}$. the first available frame is 9.

3. construct the physical address by concatenation $f|d = 0x9E6$

q4(c)

1. first find the page number and offset of the address by converting to binary then hex and split it with respect to bit widths

$0x94A \rightarrow (1001 \ 0100 \ 1010) \rightarrow (p, d) = (0x9, 0x4A)$

2. look up $p = 9$ in the table and we find that $f = 0x1$
3. construct the physical address by concatenation $f|d = 0x14A$

Question 6

q6(a)

Its a way to manage memory. Its when you only load a page when they are accessed (i.e "demanded") during the run time of a program. Pages that are not accessed will not be loaded on to physical memory and will remain on the disk. To distinguish between loaded/non-loaded pages you need some form of hardware support like a valid-invalid bit scheme.

q6(b)

reference string: $\{0,6,3,0,2,6,3,5,2,4,1,3,0,6,1,4,2,3,5,7\}$

0 miss [0] fault
 6 miss [0, 6] fault
 3 miss [0, 6, 3] fault
 0 hit [0, 6, 3] no fault
 2 miss [0, 6, 3, 2] fault
 6 hit [0, 6, 3, 2] no fault
 3 hit [0, 6, 3, 2] no fault
 5 miss [0, 6, 3, 2, 5] fault
 2 hit [0, 6, 3, 2, 5] no fault
 4 miss [6, 3, 2, 5, 4] evicted 0, fault
 1 miss [3, 2, 5, 4, 1] evicted 6, fault
 3 hit [3, 2, 5, 4, 1] no fault
 0 miss [2, 5, 4, 1, 0] 3 evicted, fault
 6 miss [5, 4, 1, 0, 6] evicted 2, fault
 1 hit [5, 4, 1, 0, 6] No fault
 4 hit [5, 4, 1, 0, 6] no fault
 2 miss [4, 1, 0, 6, 2] evicted 5, fault
 3 miss [1, 0, 6, 2, 3] evicted 4, fault
 5 miss [0, 6, 2, 3, 5] Evicted 1, fault
 7 miss [6, 2, 3, 5, 7] Evicted 0, fault
 15 page faults for first string

reference string: $\{3,1,4,2,5,4,1,3,5,2,0,1,1,0,2,3,4,5,0,1\}$

3 miss [3] fault
 1 miss [3, 1] fault
 4 miss [3, 1, 4] fault
 2 miss [3, 1, 4, 2] fault
 5 miss [3, 1, 4, 2, 5] fault
 4 hit [3, 1, 4, 2, 5] no fault
 1 hit [3, 1, 4, 2, 5] no fault
 3 hit [3, 1, 4, 2, 5] no fault
 5 hit [3, 1, 4, 2, 5] no fault

2 hit [3, 1, 4, 2, 5] no fault
 0 miss [3, 1, 4, 2, 5] evict 3, fault
 1 hit [1, 4, 2, 5, 0] no fault
 1 hit [1, 4, 2, 5, 0] no fault
 0 hit [1, 4, 2, 5, 0] no fault
 2 hit [1, 4, 2, 5, 0] no fault
 3 miss [4, 2, 5, 0, 3] evict 1, fault
 4 hit [4, 2, 5, 0, 3] no fault
 5 hit [4, 2, 5, 0, 3] no fault
 0 hit [4, 2, 5, 0, 3] no fault
 1 miss [2, 5, 0, 3, 1] evict 4, fault

8 faults

reference string {4,2,1,7,9,8,3,5,2,6,8,1,0,7,2,4,1,3,5,8}

4 miss [4] fault
 2 miss [4, 2] fault
 1 miss [4, 2, 1] fault
 7 miss [4, 2, 1, 7] fault
 9 miss [4, 2, 1, 7, 9] fault
 8 miss [2, 1, 7, 9, 8] evict 4, fault
 3 miss [1, 7, 9, 8, 3] evict 2, fault
 5 miss [7, 9, 8, 3, 5] evict 1, fault
 2 miss [9, 8, 3, 5, 2] evict 7, fault
 6 miss [8, 3, 5, 2, 6] evict 9, fault
 8 hit [8, 3, 5, 2, 6] no fault
 1 miss [3, 5, 2, 6, 1] evict 8, fault
 0 miss [5, 2, 6, 1, 0] evict 3, fault
 7 miss [2, 6, 1, 0, 7] evict 5, fault
 2 hit [2, 6, 1, 0, 7] no fault
 4 miss [6, 1, 0, 7, 4] evict 2, fault
 1 hit [6, 1, 0, 7, 4] no fault
 3 miss [1, 0, 7, 4, 3] evict 6, fault
 5 miss [0, 7, 4, 3, 5] evict 1, fault
 8 miss [7, 4, 3, 5, 8] evict 0, fault
 17 faults